

1. Принятые решения

SSTable

1. Файлы разбиты на блоки, размер которых кратен 4096 байт
2. Блоки идут в файле подряд (с учётом выравнивания)
3. Записи в блоках хранятся так:
 - длина ключа (4 байта)
 - длина значения (4 байт)
 - ключ
 - значение
4. После всех блоков идёт footer: <информация о блоках + контрольная сумма
5. информация о блоках включает в себя:
 - информация о блоках:
 - индекс первого ключа - offset (8 байт)
 - индекс последнего ключа - offset (8 байт)
 - количество блоков (8 байт)
6. Контрольная сумма записывается в следующем формате
 - контрольная сумма (X байт)
 - длина контрольной суммы (4 байта)
7. При проверке наличия ключа в блоке используется информация о первом и последнем ключе таблицы
8. При проверке наличия ключа в таблице используется бинарный поиск по блокам с учётом их первого и последнего ключей
9. Итератор ленивый, то есть он читает по одному элементу (key value) с диска

WAL

1. После каждой записи в WAL идёт контрольная сумма, дополнительных контрольных сумм нет
2. Формат хранения записей следующий:
 - тип записи (1 байт) - либо это полноценная запись, либо запись после которого точно ничего нет (guard); используется если контрольные суммы будут вычисляться для нескольких записей
 - длина ключа (4 байта)
 - ключ
 - длина значения (4 байта)

- значение
3. Формат хранения контрольной суммы следующий:
 - длина контрольной суммы (4 байта)
 - контрольная сумма
 4. Итератор ленивый, то есть он читает по одному элементу (key value) с диска

LSM

1. Все записи сохраняются в memtable (как следствие и в sstable) расширенным значением: `flag value`, где
 - `flag` - 1 байт, о значении которого можно понять, эта запись tombstone или нет
 - `value` - реальное значение записи (в случае tombstone это 0 байт)Соответственно, является запись tombstone или нет проверяется с помощью её расширенного значения, взятого из memtable или sstable
2. Все таблицы хранятся по уровням, где максимальное число таблиц на уровне $i = T$ в степени i . Как следствие, поиск осуществляется по уровням, а также итератор по хранилище учитывает уровни (чем ближе уровень к memtable, тем "свежее" записи).
3. Реализована оптимизация: удаление tombstone с последнего слоя автоматически при compaction
4. Memtable "сливается" на диск (flush), если размер его записей превышает заданный порог (сейчас - 100MB). Размер записи в memtable считается как размер ключа + размер значения (в байтах).
5. На самом первом уровне (речь о хранении на диске, не о memtable) таблицы могут пересекаться по диапазонам ключей, а на следующих уровнях уже нет. Это достигается за счёт алгоритма compaction.
6. Если на самом первом уровне превышено число таблиц, то все таблицы этого уровня сливаются с таблицами следующего (с теми, с которыми есть пересечение). А если превышено число таблиц не на первом уровне, то выбирается таблица, которая пересекается с наименьшим числом таблиц следующего уровня, и compaction происходит с ней.
7. Каждая таблица хранится на диске в отдельном файле, при этом название файла - количество наносекунд, прошедших с начала эпохи (Unix) до момента создания таблицы. Засчёт этого достигается то что после краша можно считать дату создания таблиц из названий файлов и не нужно таблицы заново пересоздавать.

LSM Iterator

- Итератор реализован как k-way итератор. То есть объект итератора хранит в себе список итераторов всех sstable и memtable, а также список полученных пар ключ-значение.

- Метод `Next()` сортирует пары по ключу в порядке возрастания и возвращает первую пару.
- В каждый момент времени размер массива пар не превышает `1 + количество всех таблиц (1 - за memtable)`. Это достигается за счёт того, все итераторы таблиц после получения следующего элемента становятся заблокированными, то есть из них не будут получаться пары. Итератор таблицы становится разблокированным, если из массива пар удалена пара, полученная итератором (при возврате пользователю); либо если в паре массива, полученной итератором, заменено значение. Последнее может произойти, так как в общем случае таблицы могут пересекаться по ключам, поэтому в массиве пар нужно хранить самое свежее значение ключа.

2. Контроль (как проверялось)

Тесты

- Есть тесты на `sstable` - запись, чтение, итерация (в том числе с граничными случаями)
- Есть тесты на `WAL` - запись, чтения, итерация (в том числе с граничными случаями)
- Есть тесты на `LSM` - запись, чтение, удаление, итерация (в том числе с граничными случаями).

Каждый тест `LSM` включает в себя проверку до "краша" и после. При этом все `LSM` тесты проверялись на двух конфигурациях:

- `MemtableFlushThreshold = 0, T = 2;`
- `MemtableFlushThreshold = 1024 * 1024, T = 10;`

Бенчмарки

Было добавлено 5 бенчмарок для `LSM`:

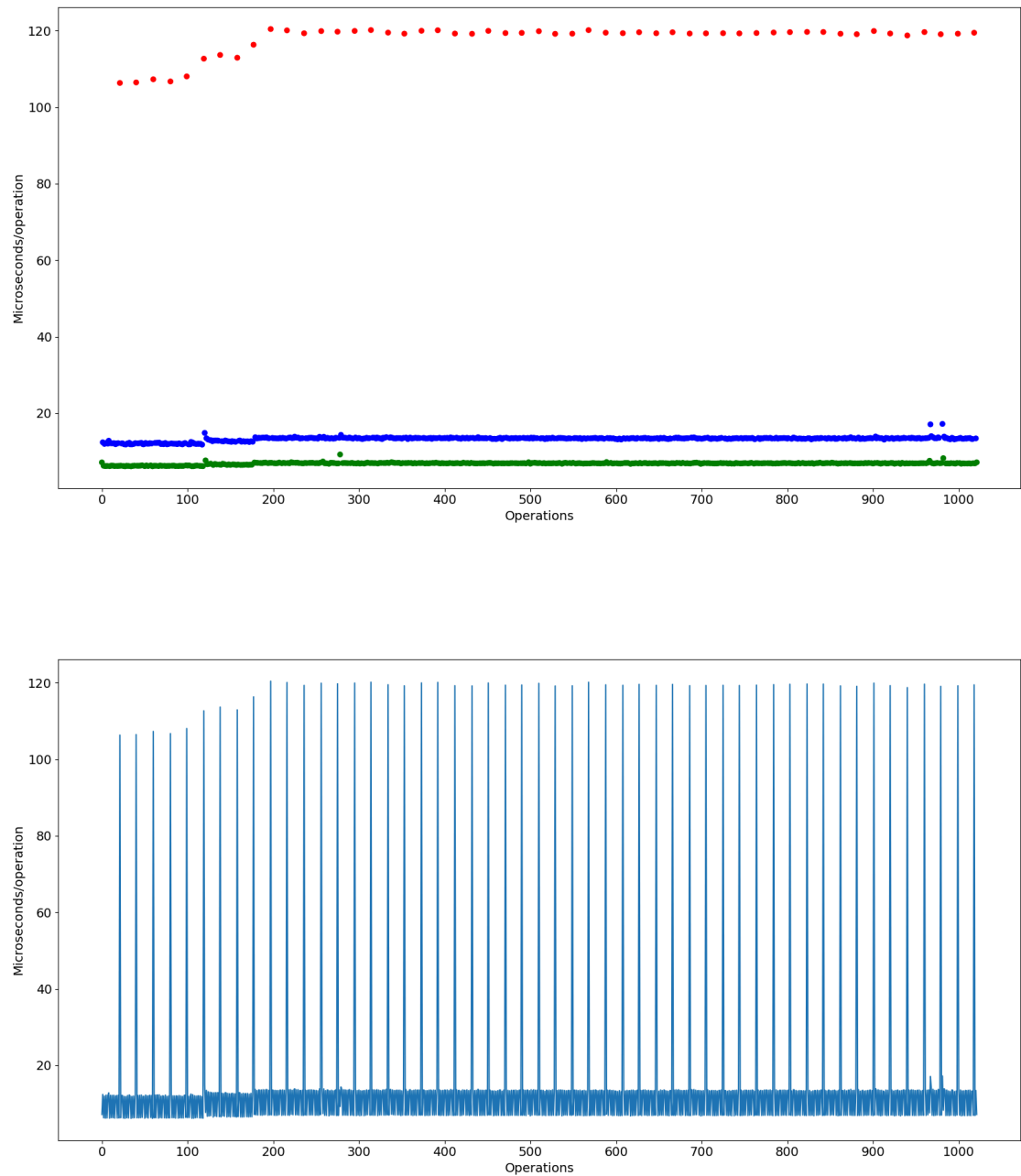
- *LargeAllUnique* - запись (только `Put`) `256 x 256 x 256 x 4` уникальных пар с длиной ключа и значения - 4 байта каждое
- *Large256Unique* - запись (только `Put`) `256 x 256 x 256 x 4` пар с длиной ключа 1 байт и значения - 4 байта (то есть уникальных ключей всего 256)
- *CDR* (Call Detail Record) - запись (только `Put`) записей из `JSON`-файла с 5 миллионов записей, сгенерированных скриптом `gen_cdr.py`. Ключ - `imsi, timestamp`, значение - `msisdn, call_type, duration, cell_id, lat, lon`
- *PutN* - 20 итераций записи (только `Put`), при этом на каждой итерации записывается `5000 x номер итерации` уникальных записей; каждая итерация повторялась 5 раз для большей стабильности результатов
- *PutGetN* - то же самое, что предыдущий бенчмарки, но количество на каждой итерации менялось, плюс к `Put` с некоторой вероятностью добавлялся вызов `Get`.

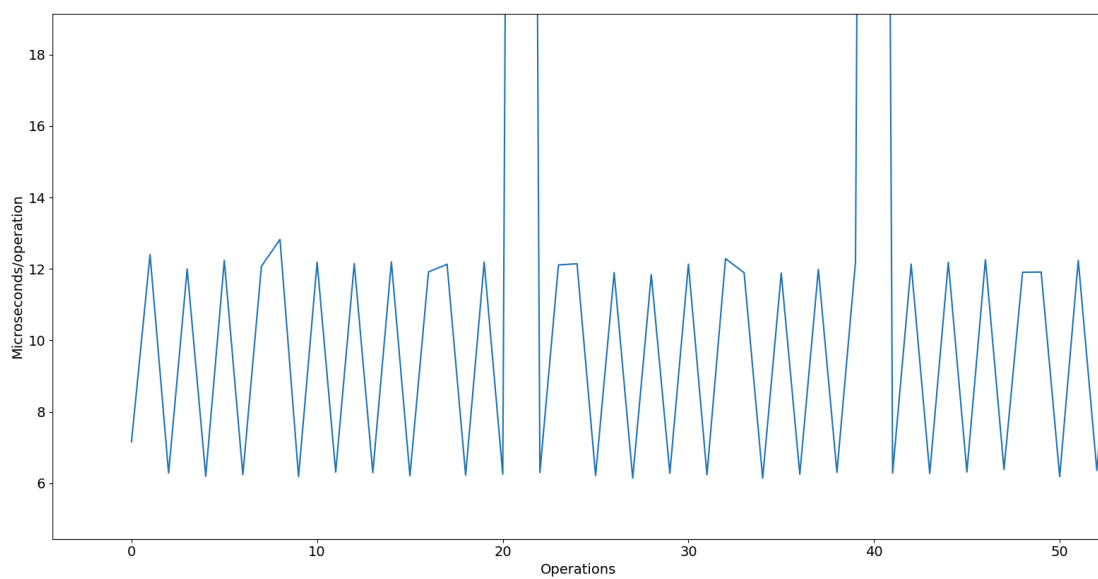
3. Результаты

Все бенчмарки запускались с MemtableFlushThreshold = 1M и T = 10.

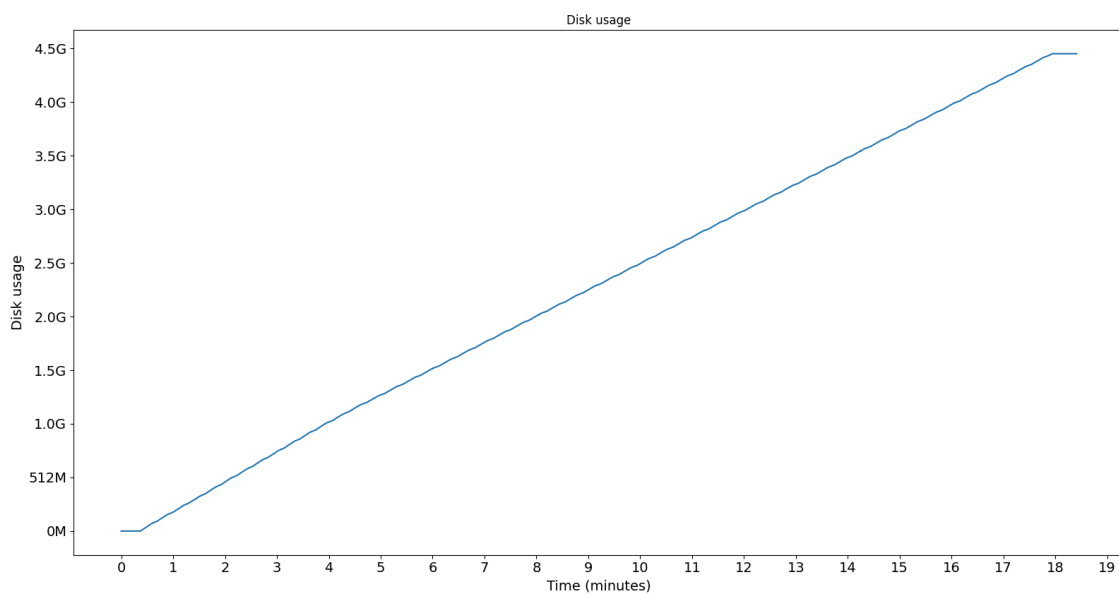
LargeAllUnique

Зависимость времени операций от проведённых операций

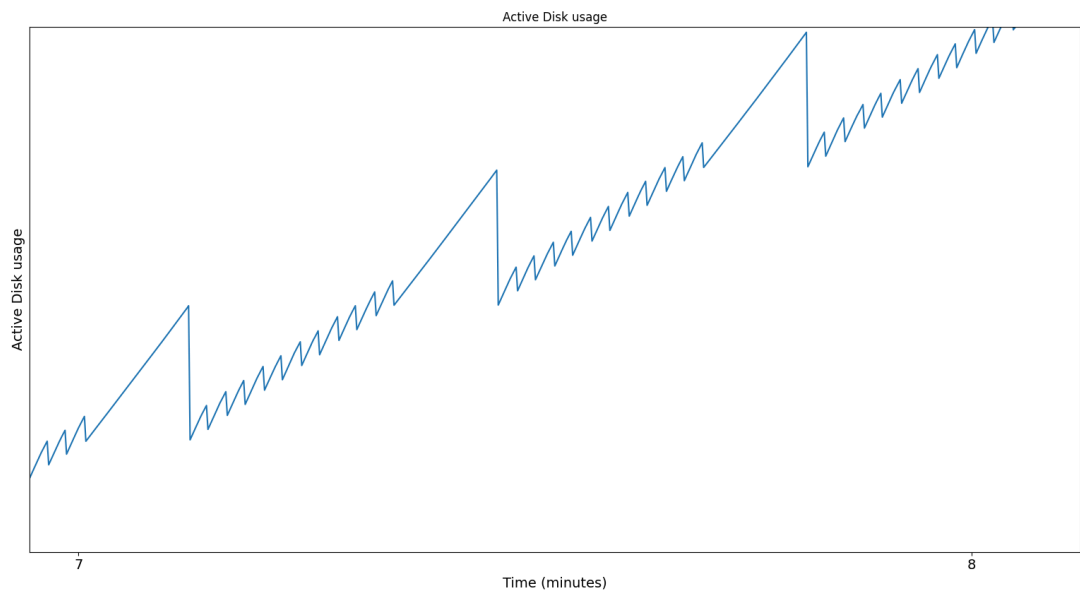
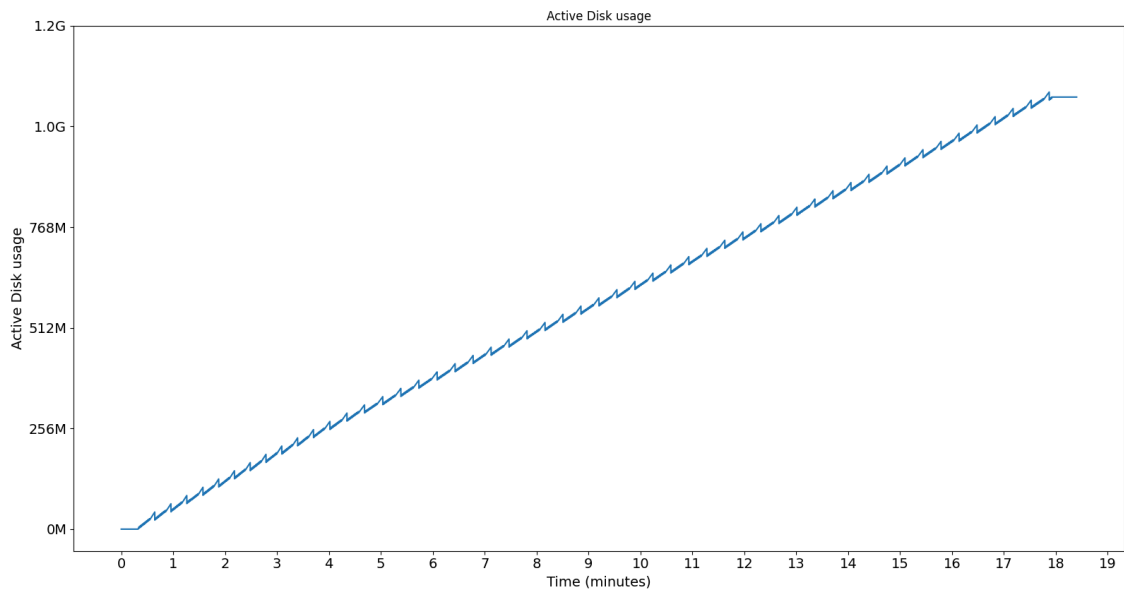




Полное потребление диска во времени

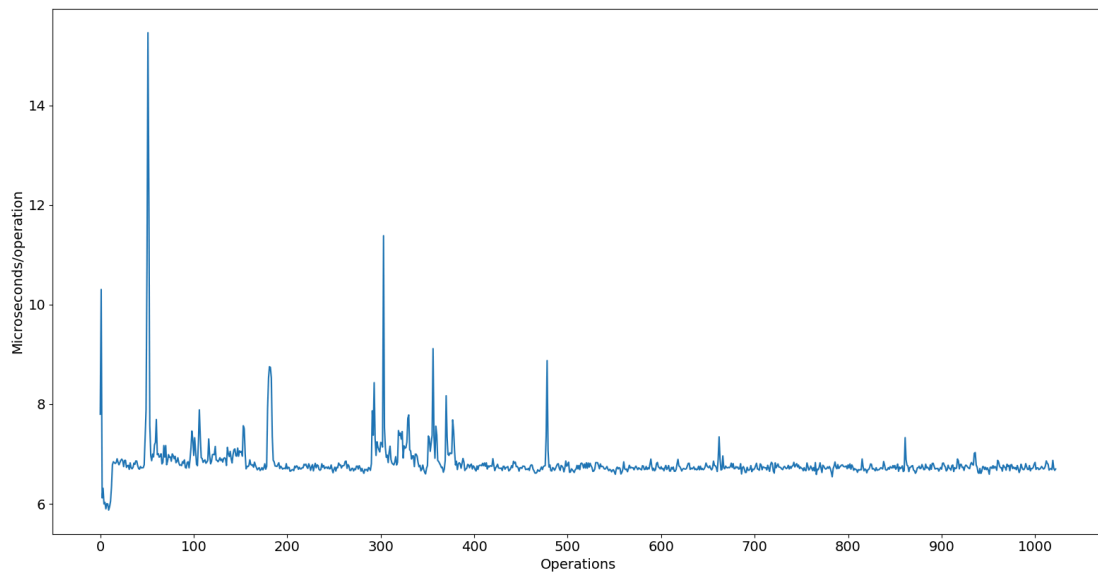


Активное потребление диска во времени

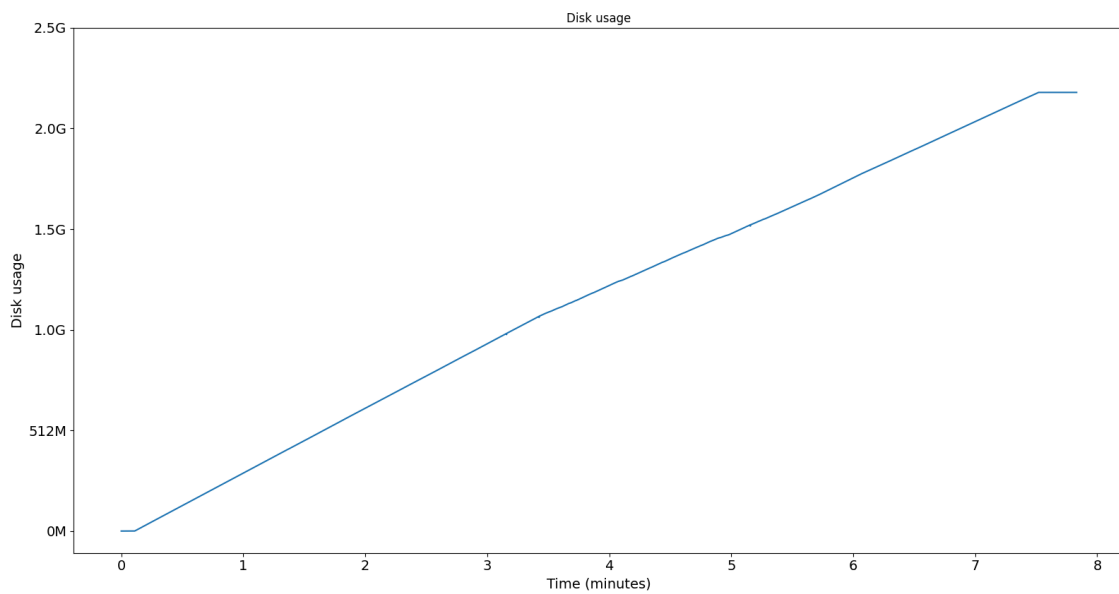


Large256Unique

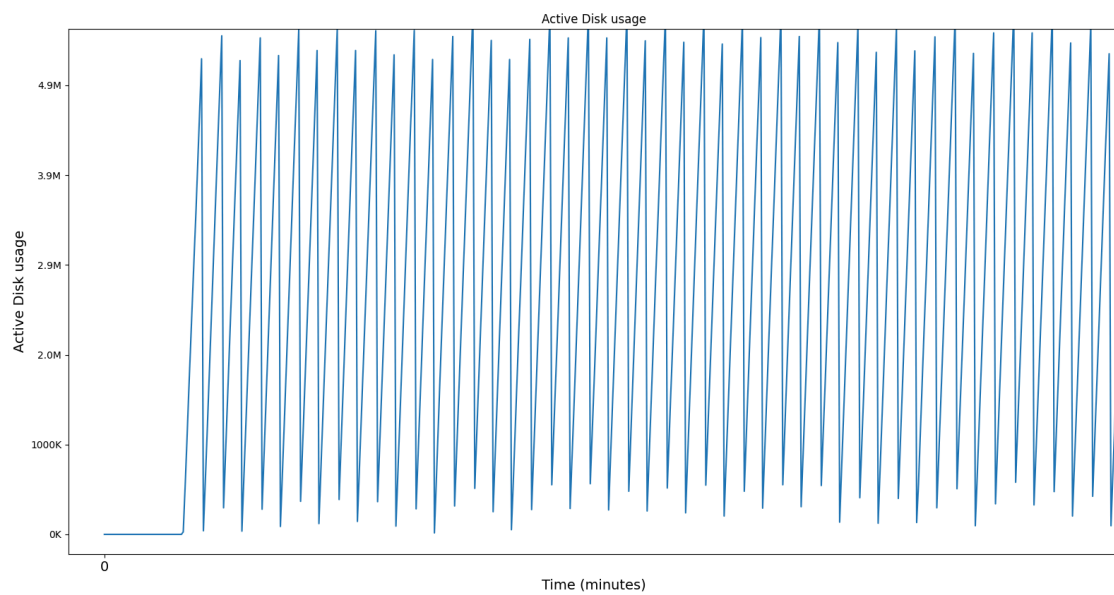
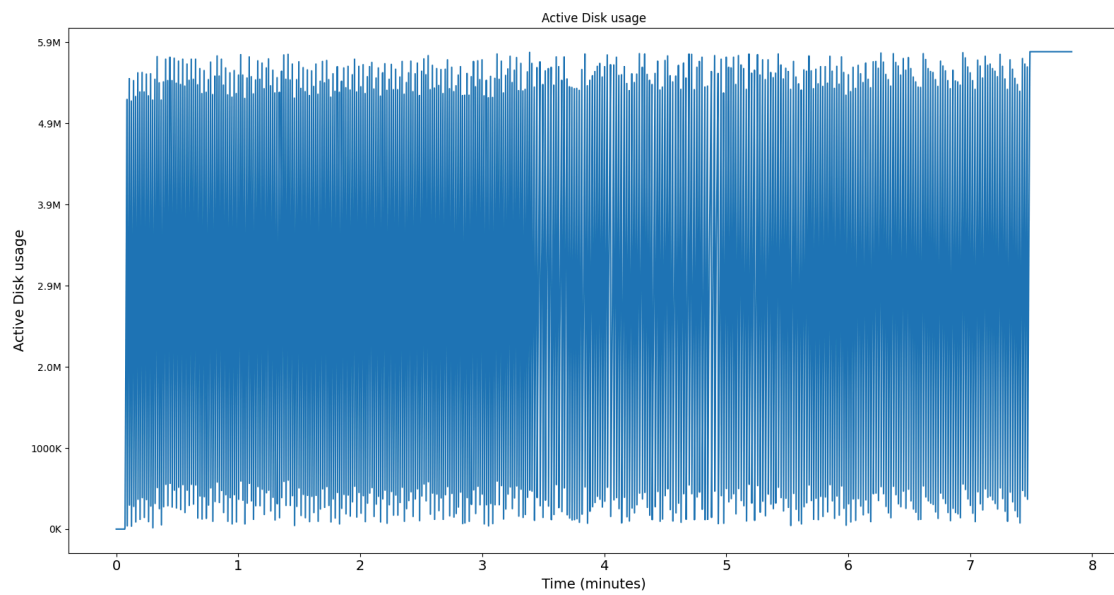
Зависимость времени операций от проведённых операций

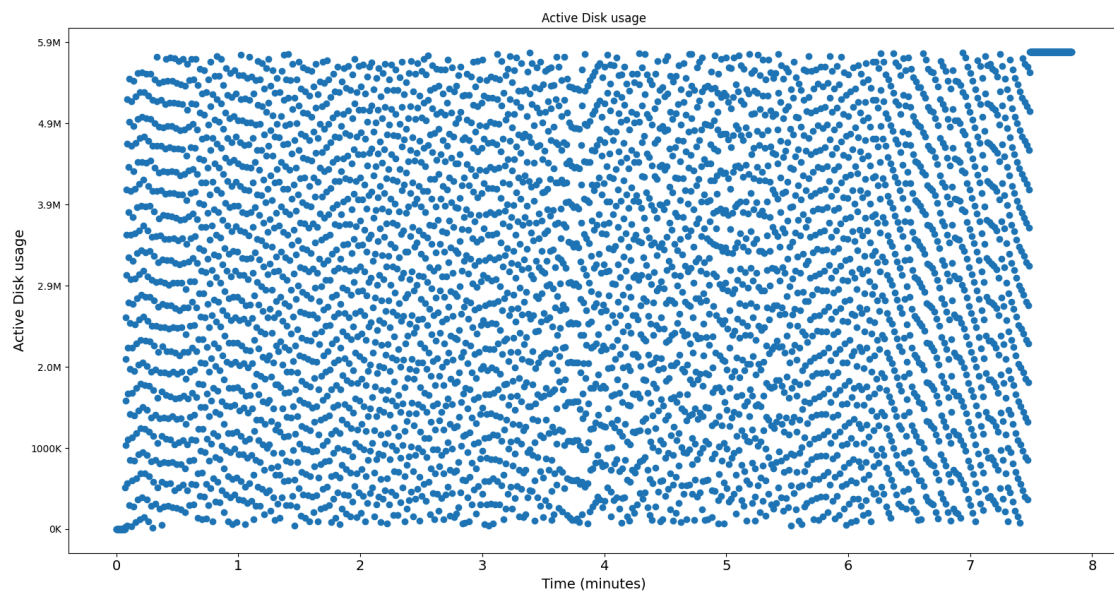


Полное потребление диска во времени



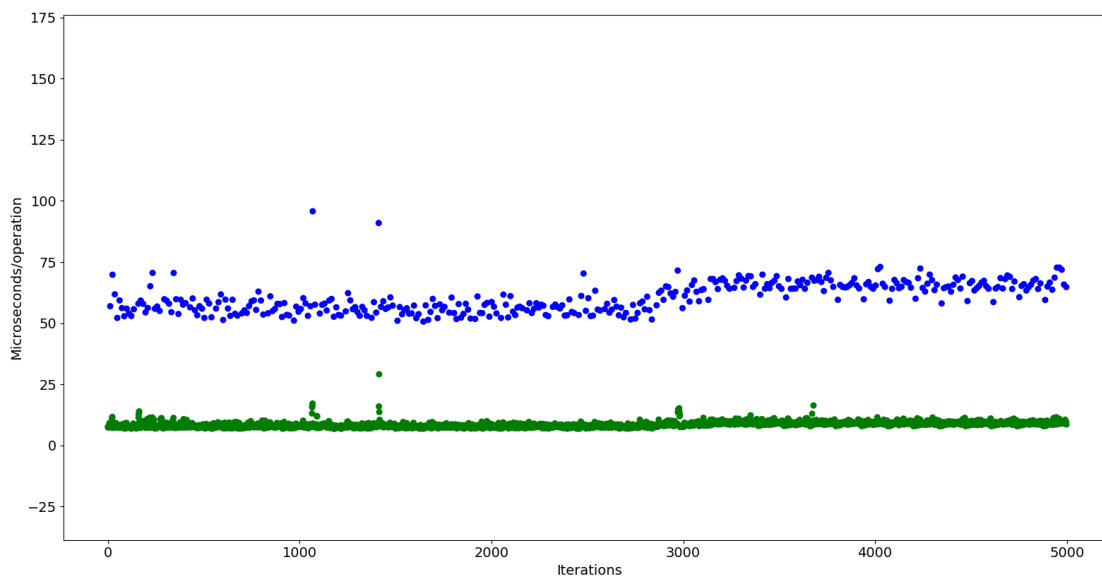
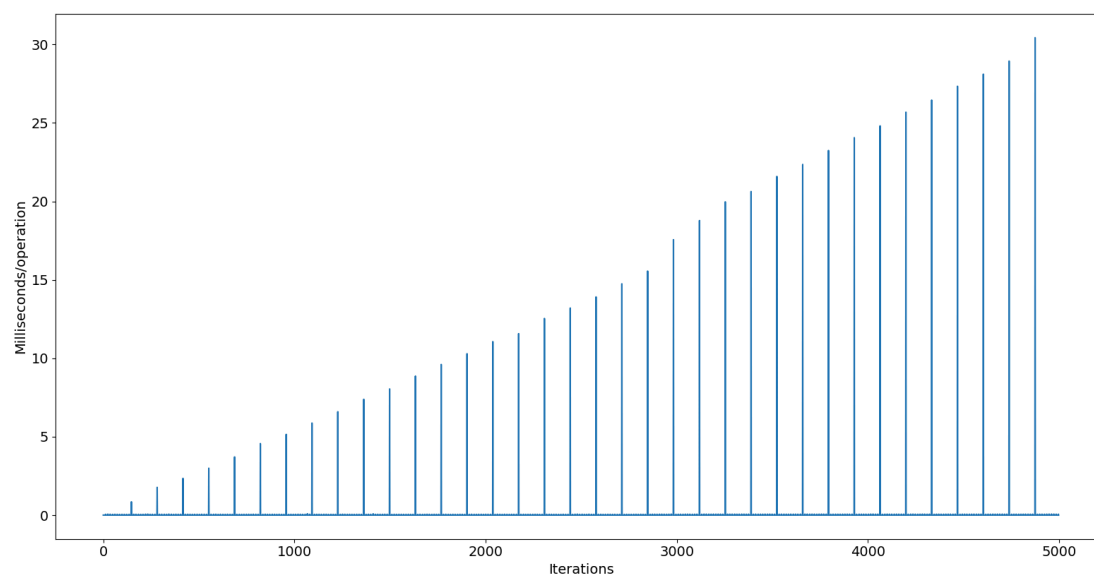
Активное потребление диска во времени



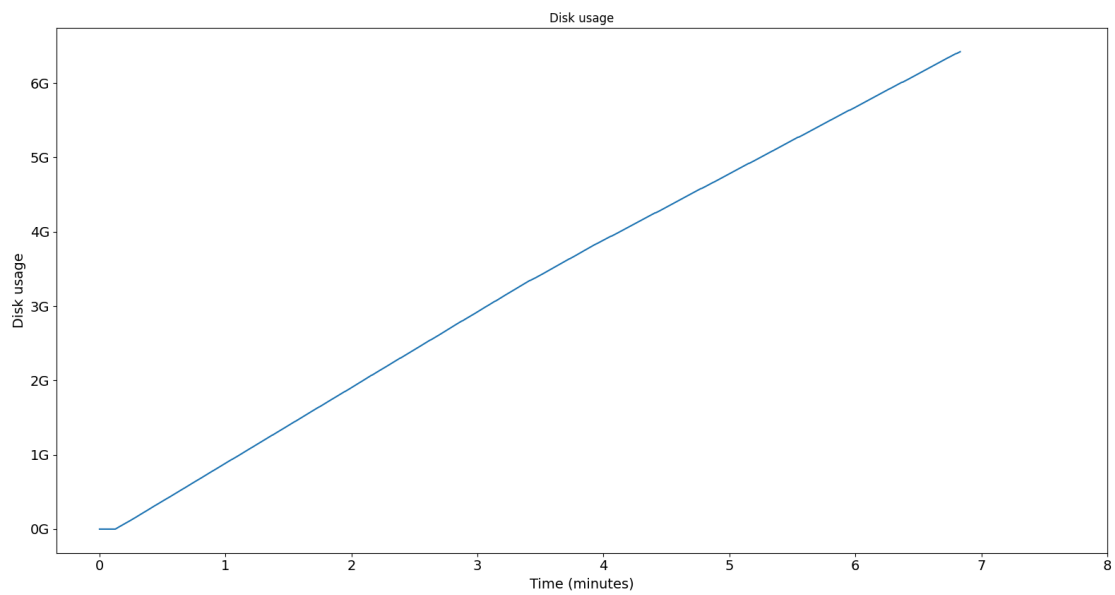


CDR

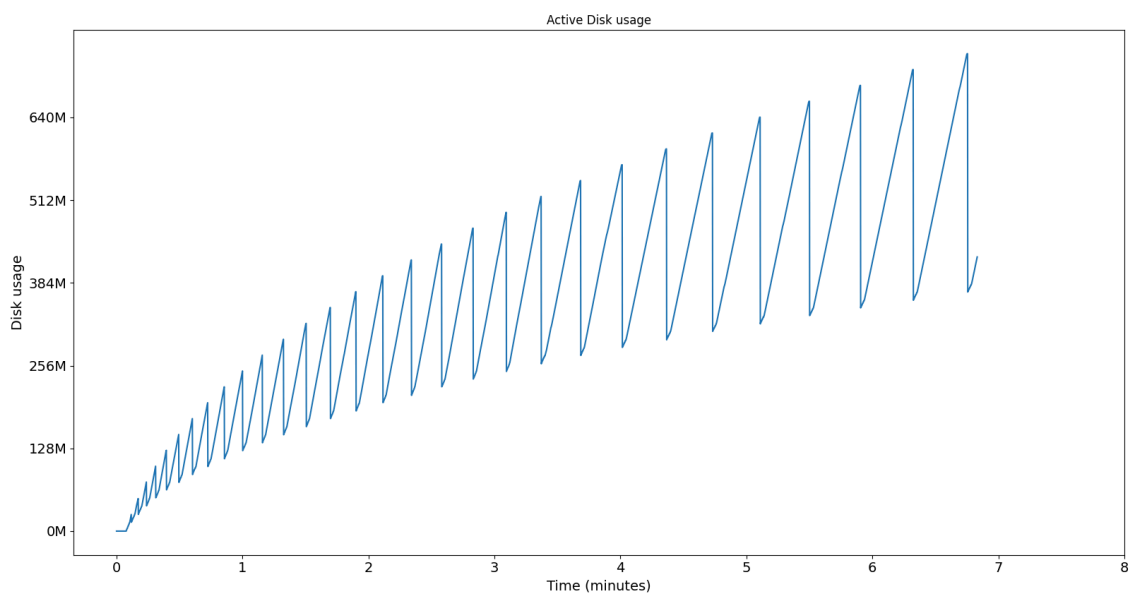
Зависимость времени операций от номера итерации (итерация - запись 1000 элементов)



Полное потребление диска во времени

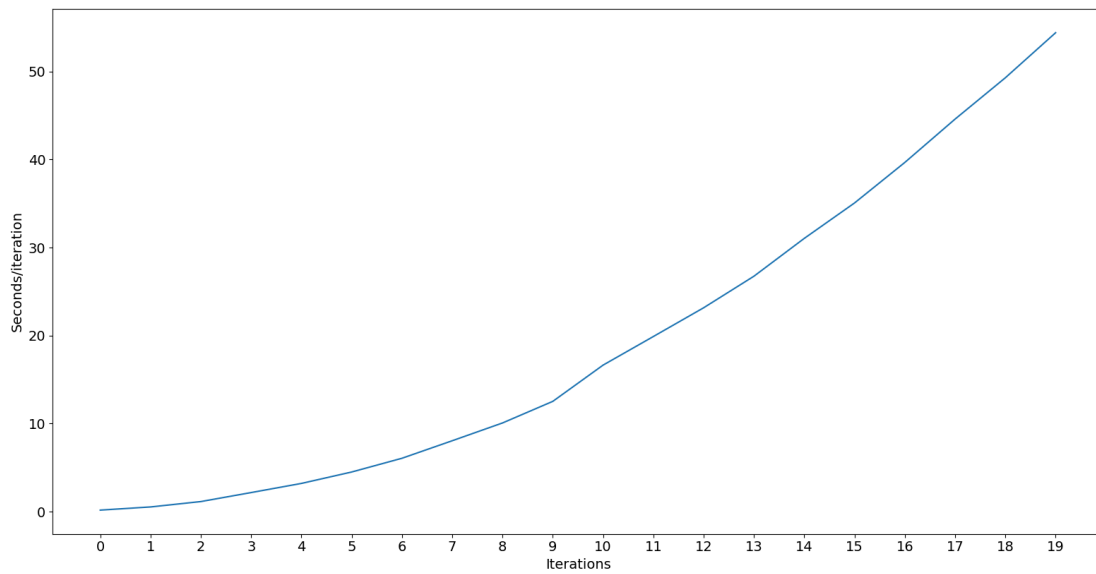


Активное потребление диска во времени



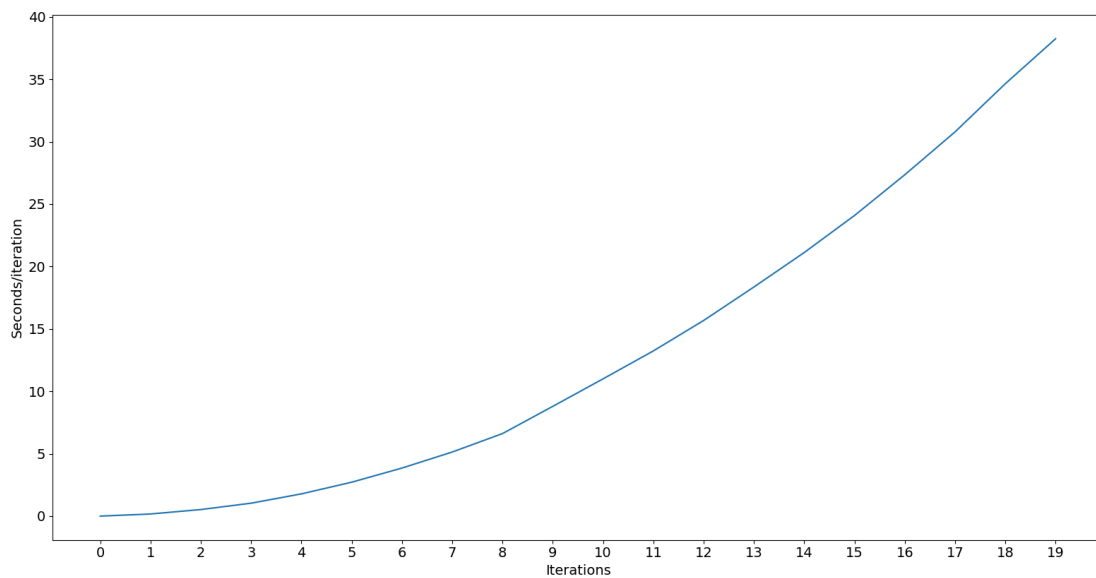
PutN ($i * 5000$)

Зависимость времени операций от номера итерации



PutGet0.01 ($i * 4000$, Get с вероятностью 0.01)

Зависимость времени операций от номера итерации



PutGet0.5 ($i * 500$, Get с вероятностью 1/2)

Зависимость времени операций от номера итерации

